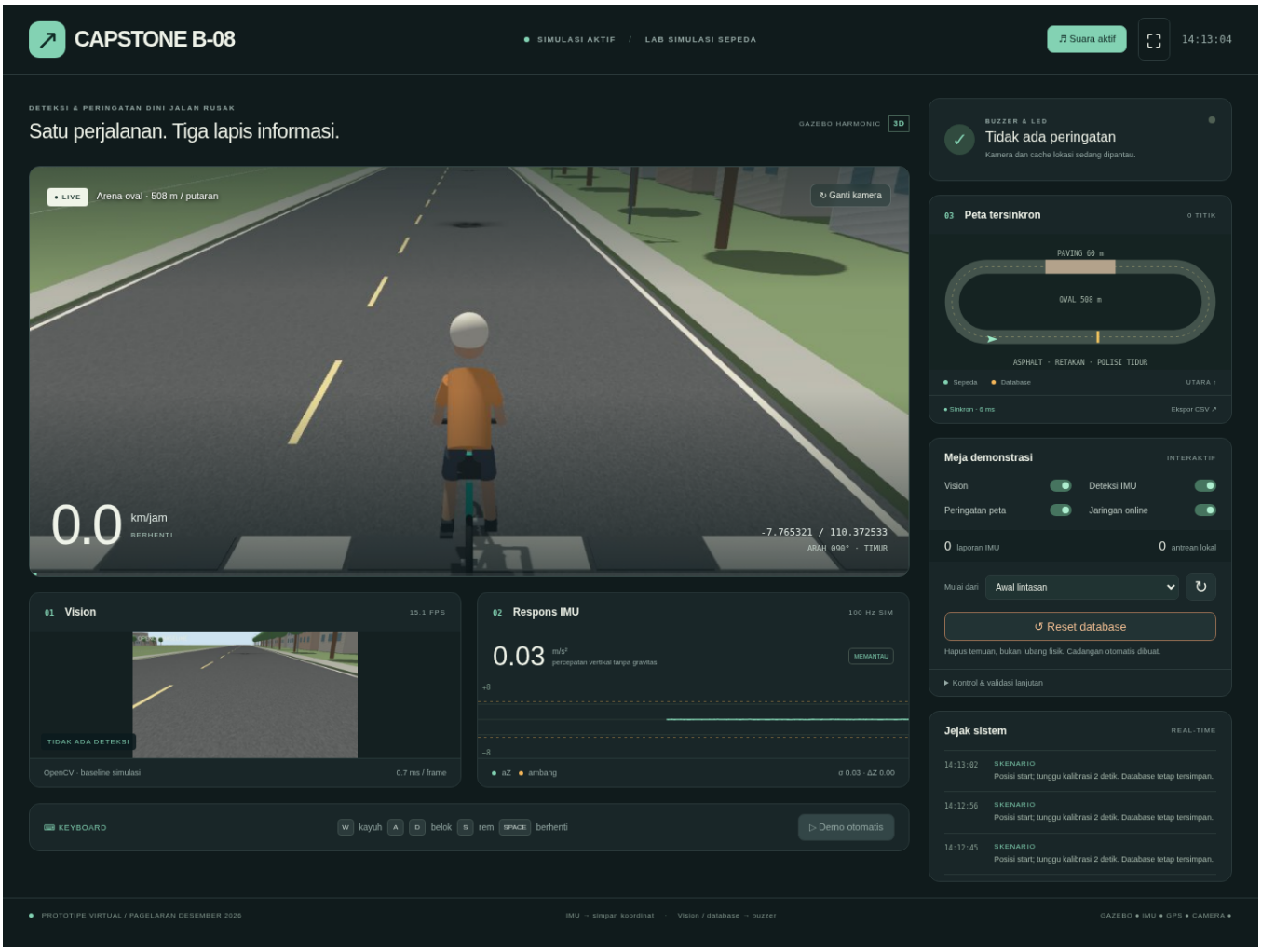
# Jaga Jalan
# Panduan simulasi

Deteksi jalan rusak berbasis IMU dan vision untuk pengendara sepeda.



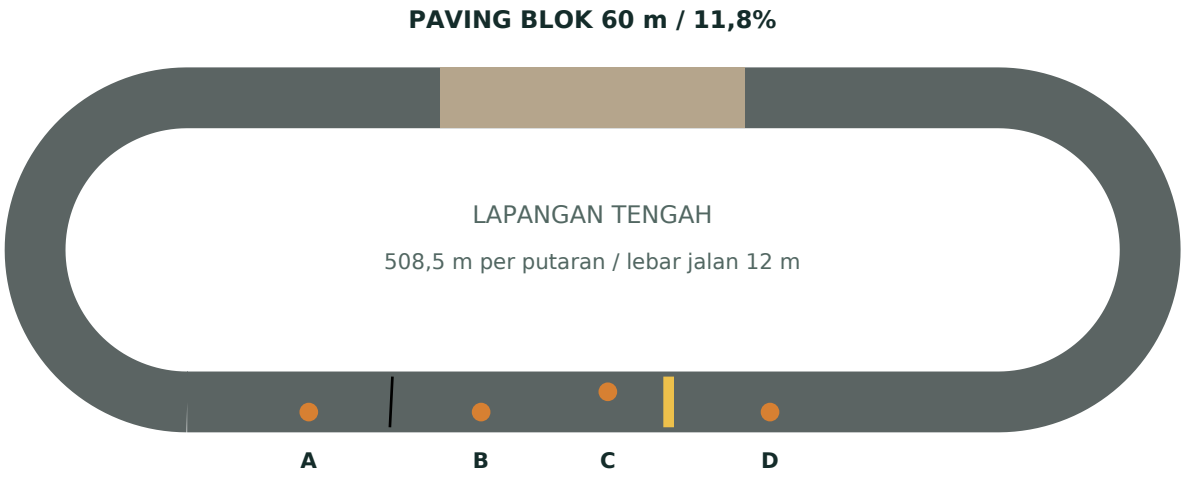

## Jalankan: bash start.sh · Buka: http://localhost:8765

Klik Aktifkan suara, tunggu kalibrasi, lalu gunakan W/A/S/D atau stik PS. Panduan ini mencakup skenario pagelaran, alur data, parameter, dan pemecahan masalah.

---

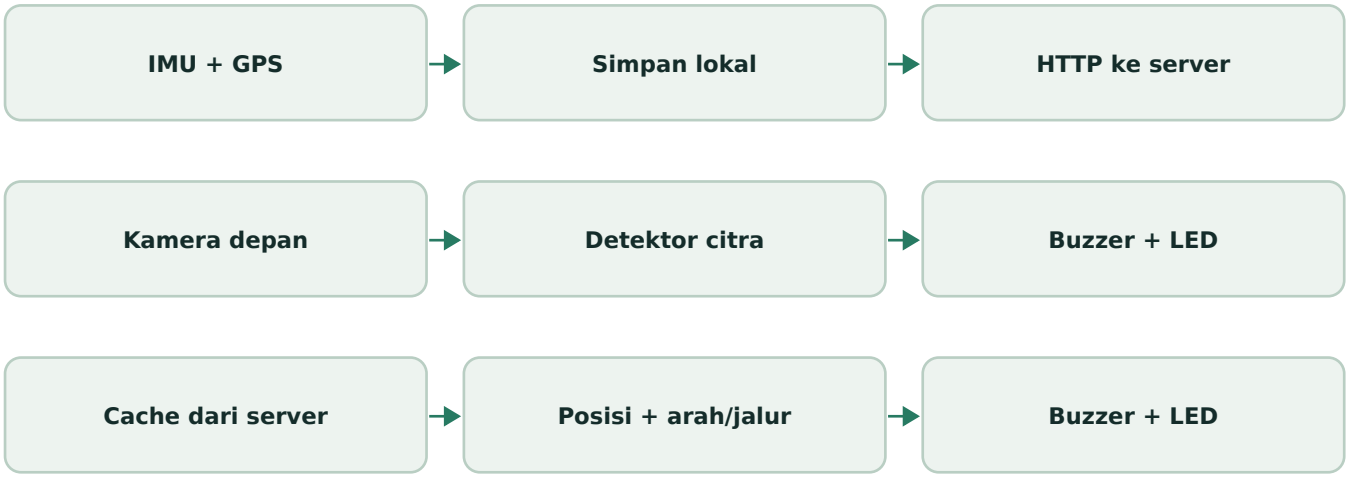
Edisi arena oval, 22 Oktober 2026. Berdasarkan proposal C-251 B-08 revisi 01 dan pembaruan kebutuhan pengguna. Gazebo Harmonic, paving 60 m, retakan, polisi tidur, pothole, dan reset database. Detektor OpenCV adalah baseline; bobot FOMO belum disertakan.

# Arena oval dan permukaan pembanding




Skema objek dunia, bukan isi database: A–D = pothole. Retakan pada x=40 m; garis kuning x=95 m = polisi tidur. Aspal mendominasi lintasan. Peta dashboard hanya menampilkan titik pothole yang sudah tersinkron.

# Arsitektur demonstrasi




Penyimpanan perangkat dan server dipisahkan. Saat offline, temuan IMU tetap masuk antrean lokal; peringatan peta memakai cache terakhir. Keputusan vision menggunakan piksel kamera.

# Isi panduan

1. Tujuan dan hasil yang ditampilkan
2. Mulai dalam beberapa menit
3. Pilihan menjalankan
4. Keyboard, stik PS, dan kontrol layar
5. Membaca dashboard
6. Naskah demonstrasi untuk pengunjung

7. Bagaimana simulasi bekerja

8. Berkas proyek dan parameter

9. Memasang pada komputer lain

10. Pemecahan masalah

11. Pengujian dan batas interpretasi

12. Persiapan hari pagelaran

Referensi

# 1. Tujuan dan hasil yang ditampilkan

Jaga Jalan adalah demonstrasi sistem deteksi dan peringatan dini jalan rusak bagi pengendara sepeda. Pengunjung mengendarai sepeda virtual di Gazebo melalui keyboard atau stik PS. Dashboard memperlihatkan kamera depan, kamera mengikuti sepeda, grafik IMU, posisi GPS, peta lokasi tersinkron, antrean laporan, dan alasan buzzer menyala.

Simulasi mengikuti tiga alur: IMU mengklasifikasikan pola pothole lalu menyimpan koordinat ke server; vision mendeteksi dugaan pothole hanya untuk menyalakan buzzer; database memberi peringatan ketika sepeda mengarah dan mendekat ke titik tersimpan. Vision tidak menyimpan laporan dan tidak menjadi syarat laporan IMU. Saat offline, laporan IMU masuk antrean lokal.

Edisi arena oval: lintasan loop sekitar 508,5 m, terdiri dari dua lintasan lurus masing-masing 160 m dan dua setengah lingkaran berjari-jari 30 m. Lebar jalan 12 m. Sebagian besar jalan adalah aspal; paving blok hanya 60 m (11,8% panjang lintasan), ditambah retakan, polisi tidur, dan empat pothole. Target deteksi adalah pothole, bukan seluruh jenis ketidakrataan jalan.

Referensi utama: B-08_Naskah Proposal C251 Final.pdf, Dokumen C-251 revisi 01, 14 Agustus 2026. Rujukan alur sistem terdapat pada Bab 7 halaman dokumen 62–63 (halaman PDF 78–79). Spesifikasi keluaran pada Tabel 6.2, halaman dokumen 54 (PDF 70).

# 2. Mulai dalam beberapa menit

Buka terminal pada folder proyek. Jalankan:

```
cd /home/shane/Capstone/simulasi_pothole
bash start.sh
```

Launcher mengompilasi plugin jika diperlukan, membuat aset jalan lokal, lalu menjalankan tiga proses: Gazebo, server database, dan perangkat/dashboard. Tunggu hingga alamat dashboard muncul. Saat pertama kali memuat kamera, waktu tunggu dapat mencapai 20–40 detik, tergantung GPU dan CPU.

1. Buka http://localhost:8765 pada Firefox atau Chrome di komputer yang sama.

2. Pastikan indikator GAZEBO, IMU, GPS, dan CAMERA aktif. Tunggu kalibrasi IMU selesai selama sekitar dua detik waktu simulasi.

3. Klik Aktifkan suara. Browser memerlukan klik pengguna sebelum dapat memutar buzzer.

4. Klik area judul atau gambar agar keyboard tidak sedang memilih menu.

5. Tekan W untuk mengayuh, A/D untuk membelok, dan spasi untuk berhenti. Atau klik Demo otomatis untuk menyusuri lajur demonstrasi dengan target sekitar 10,8 km/jam.

6. Tekan Ctrl+C pada terminal launcher untuk mengakhiri demonstrasi. Database dan log tetap tersimpan.

Server lokal memakai port 8766, sedangkan dashboard memakai 8765. Ini adalah server simulasi pada laptop, bukan layanan cloud yang sudah dipublikasikan. Internet tidak dibutuhkan setelah dependensi tersedia.

# 3. Pilihan menjalankan

| Kebutuhan | Perintah dari folder proyek |
|---|---|
| Dashboard dan sensor Gazebo | bash start.sh |
| Tambahkan jendela 3D Gazebo | bash start.sh --gui |
| Rendering tanpa jendela, melalui EGL | bash start.sh --headless |
| Database baru tanpa menghapus data lama | bash start.sh --data-dir data/pagelaran-01 |
| Database awal kosong | bash start.sh --no-seed --data-dir data/kosong-01 |
| Hindari port yang sudah dipakai | bash start.sh --port 8775 --server-port 8776 |

Nama folder data dapat diganti untuk setiap sesi pagelaran. Folder yang sudah pernah dipakai melanjutkan database sesi itu. Untuk mengulang penemuan lubang tanpa memulai ulang aplikasi, klik **Reset database** di Meja demonstrasi lalu konfirmasi. Tombol panah di sebelah pilihan skenario hanya memindahkan sepeda; tombol itu tidak menghapus database.

Reset database mengosongkan server, cache perangkat, laporan lokal, dan antrean offline. Sepeda direm, demo otomatis dihentikan, dan IMU dikalibrasi ulang. Lubang fisik di arena tetap ada untuk ditemukan kembali. Setelah reset, titik contoh B tidak dimuat lagi ketika aplikasi dinyalakan ulang. Tombol tetap dapat digunakan saat sakelar Jaringan online mati, karena reset merupakan perintah operator langsung ke server lokal. Jika proses server tidak tersedia, perintah gagal dan data lokal tidak dihapus.

Sebelum pengosongan, salinan database dibuat pada folder **backups/** di folder data aktif. Nama berkas mencantumkan device/server, waktu, dan ID unik. Cadangan ini tetap berisi data lama; reset bukan penghapusan permanen arsip. Untuk memulihkan, hentikan launcher lalu gunakan pasangan cadangan device dan server dari operasi yang sama pada folder sesi baru. Jangan menimpa database saat proses masih berjalan. Log events.jsonl tetap dipertahankan untuk audit.

Opsi --gui menambahkan jendela Gazebo untuk pengamatan dunia dari sudut bebas. Kontrol utama tetap dilakukan dari dashboard. Gunakan satu tab dashboard sebagai pengendali agar input antar-tab tidak saling menggantikan.

# 4. Keyboard, stik PS, dan kontrol layar

| Aksi | Keyboard | Stik standar / layar |
|---|---|---|
| Kayuh maju | W atau panah atas | R2 |
| Belok kiri/kanan | A/D atau panah kiri/kanan | Analog kiri horizontal |
| Rem | S atau panah bawah | L2 atau tombol silang (X) |
| Berhenti | Spasi | Tombol kotak pada kontrol sentuh |
| Mundur pelan | Shift + S | Keyboard |
| Kembali ke awal | R | Segitiga |
| Demo otomatis | P | Options / tombol Demo otomatis |
| Pindah titik awal skenario | Pilih menu lalu tombol panah melingkar | Dashboard |

Untuk stik PS4/PS5, sambungkan terlebih dahulu melalui USB atau Bluetooth pada sistem operasi. Buka dashboard pada localhost, lalu tekan tombol stik agar browser mengungkap perangkat melalui Gamepad API. Indikator kiri bawah berubah menjadi STIK PS / GAMEPAD.

Analog mempunyai dead zone 12 persen untuk mengurangi gerak karena drift. Jika arah stik terbalik, buka Kontrol & validasi lanjutan dan aktifkan Balik arah stik. Pemetaan R2/L2 mengikuti layout standar browser; dongle atau driver nonstandar dapat memetakan tombol secara berbeda. Pengujian perangkat PS fisik perlu dilakukan pada stik yang akan dipakai di pagelaran.

Saat fokus halaman hilang, tombol dilepas dan rem dikirim. Plugin juga mengerem jika perintah perangkat tidak diterima selama 0,6 detik. Demo otomatis memerlukan heartbeat browser; bila heartbeat hilang lebih dari 0,8 detik, mode otomatis dihentikan. Pergantian skenario menahan sepeda selama kalibrasi. Pengendali mengerem bila sepeda lebih dari 6,6 m dari garis tengah oval, tetapi masih ada jarak pengereman. Demo otomatis mengikuti loop terus-menerus pada lajur kanan; pengunjung dapat mengambil alih dengan keyboard atau stik.

# 5. Membaca dashboard

Gambar besar berasal dari kamera mengikuti sepeda. Tombol Ganti kamera memindahkannya ke kamera depan. Angka kecepatan dan arah menunjukkan gerak model; koordinat berasal dari sensor NavSat. Garis kemajuan menunjukkan posisi pada satu putaran oval dan kembali ke awal setelah melewati garis start.

Panel Vision memperlihatkan hasil pengolahan citra. Kotak kuning menandai kandidat lubang. Label OpenCV baseline selalu ditampilkan. FPS adalah laju pemrosesan menurut waktu nyata; nilai ms/frame adalah waktu komputasi detektor dan anotasi di Python, bukan latency keseluruhan sampai suara terdengar.

Panel Respons IMU menampilkan percepatan vertikal setelah gravitasi dikompensasi. Garis putus-putus menunjukkan ambang amplitudo. Amplitudo saja tidak lagi cukup untuk melaporkan pothole: detektor memeriksa urutan penurunan dan benturan. STDEV dan perubahan antarsampel tetap ditampilkan sebagai informasi. Laju 100 Hz adalah target waktu simulasi; pada komputer lambat, jumlah sampel per detik waktu nyata akan lebih kecil.

Peta tersinkron menampilkan lokasi yang sudah diterima dari server. Panah hijau menunjukkan posisi sepeda; titik kuning merupakan database. Lintasan dibuat sebagai peta lokal agar tidak membutuhkan internet atau tile peta. Koordinat acuan dekat UGM dipakai sebagai konversi geografis sintetis; jalan yang digambar bukan survei kampus yang sebenarnya.

Panel Buzzer & LED memperlihatkan sumber peringatan: VISION, DATABASE, atau keduanya. Nada vision sekitar 1080 Hz; nada database sekitar 780 Hz. Jika keduanya aktif, nada vision diprioritaskan dan kedua sumber tetap ditulis. IMU memicu penyimpanan, bukan buzzer secara langsung.

Meja demonstrasi menyediakan sakelar vision, IMU, peringatan database, dan jaringan. Angka laporan IMU adalah jumlah observasi lokal; jumlah titik peta dapat lebih kecil karena server menggabungkan observasi yang berdekatan. Antrean lokal menunjukkan laporan yang belum diakui server. Ekspor CSV mengunduh isi cache peta, bukan semua sampel sensor.

# 6. Naskah demonstrasi untuk pengunjung

## A. Vision memberi peringatan sebelum benturan

1. Klik Reset database dan konfirmasi, atau gunakan folder data baru tanpa seed.

2. Aktifkan Vision dan Deteksi IMU. Pilih A · Lubang baru, lalu klik tombol pindah skenario.

3. Tunggu kalibrasi selesai, kemudian kayuh atau aktifkan demo otomatis.

4. Kamera memberi kotak deteksi ketika lubang masuk area dekat sekitar tiga meter. Buzzer berbunyi dan panel menyebut VISION.

5. Lanjutkan melintasi lubang. Grafik IMU menunjukkan guncangan, lalu jejak sistem mencatat koordinat lokal dan konfirmasi SYNC.

6. Titik baru muncul di peta. Jelaskan bahwa vision memberi peringatan lebih awal, sedangkan IMU baru mengamati benturan setelah roda melewati kerusakan.

## B. Database memberi peringatan tanpa vision

1. Matikan Vision dan biarkan Peringatan peta aktif.

2. Pilih B · Lokasi dari server, lalu mulai bergerak.

3. Buzzer berbunyi sebelum sepeda mencapai B. Panel menampilkan DATABASE, jarak, dan estimasi waktu menuju lokasi.

4. Pada sesi baru dengan seed, B dimuat sebagai contoh laporan pengguna lain. Setelah Reset database, B kosong: lintasi B sekali dengan IMU aktif, tunggu sinkronisasi, lalu ulangi pendekatan untuk memperlihatkan peringatan database.

## C. Mendekat berbeda dari sekadar berada dekat

Dengan vision dimatikan, pilih B · Lubang di belakang, lalu kayuh maju: database tidak seharusnya memperingatkan lubang yang sedang ditinggalkan. Ulangi pada B · Lajur sebelah: titik berdekatan tetapi tidak berada pada jalur sepeda. Pada B · Mendekat dari arah balik, kayuh manual ke arah lubang; peringatan database kembali muncul. Demo otomatis dirancang mengikuti lajur maju dan tidak digunakan untuk skenario arah balik atau lajur sebelah.

### D. Koneksi terputus, laporan tetap tersimpan

1. Tunggu setidaknya satu sinkronisasi, lalu matikan Jaringan online.

2. Pilih D · Uji saat offline. Aktifkan IMU dan kayuh melewati lubang D.

3. Antrean lokal bertambah. Cache peta sebelumnya tetap dapat dipakai untuk peringatan.

4. Aktifkan Jaringan online kembali. Dalam beberapa detik antrean dikirim, server mengakui laporan, dan titik baru masuk cache.

5. Jelaskan bahwa sakelar ini mensimulasikan koneksi perangkat, sedangkan server lokal tetap berjalan.

### E. Pembanding permukaan dan pengunjung berikutnya

Pilih Retakan, Polisi tidur, atau Paving blok pada menu skenario. Kayuh melewatinya: permukaan ini bukan target pothole dan tidak seharusnya menghasilkan laporan ataupun buzzer vision. Getaran kecil tetap boleh tampak di grafik. Peringatan database dapat tetap menyala bila ada titik tersimpan lain di jalur depan; matikan Peringatan peta untuk mengisolasi uji permukaan.

Untuk membuktikan bahwa vision tidak menulis database, reset database, matikan Deteksi IMU, aktifkan Vision, lalu dekati A. Buzzer vision boleh berbunyi, tetapi jumlah laporan dan titik tetap nol. Aktifkan IMU lagi untuk sesi berikutnya. Klik Reset database dan pilih Awal lintasan untuk memberi pengalaman penemuan pertama kepada pengunjung berikutnya.

Permintaan terbaru ini menggantikan opsi validasi ganda proposal: sakelar IMU + vision telah dihapus. Jalur penyimpanan IMU dan peringatan vision sekarang independen.

## 7. Bagaimana simulasi bekerja

### Dunia dan sepeda

Lintasan mempunyai empat cekungan mesh bertekstur. Pusat A ada di (24, −2) m dengan kedalaman 12 cm; B di (58, −2) m, 17 cm; C di (83, 2) m, 9 cm; D di (115, −2) m, 15 cm. Lebar total jalan 12 m. Pohon, bangunan, trotoar, marka, lampu jalan, sepeda, dan pengendara dibuat secara prosedural dan disimpan lokal.

Oval mengelilingi lapangan terbuka. Lintasan bawah berada pada y=0 dan lintasan atas y=60 m. Paving berada pada lintasan atas, x=50–110 m, dengan pola blok 40 × 25 cm. Kontak fisik paving disederhanakan menjadi deretan balok dengan celah sempit agar triangulasi mesh tidak menimbulkan guncangan palsu. Retakan rambut di sekitar x=40 m merupakan tekstur tipis, bukan cekungan besar. Polisi tidur di x=95 m memiliki profil sinus-kuadrat sepanjang 3 m dan tinggi 8 cm, dengan collision mesh yang benar-benar mengangkat roda. Retakan dan paving bukan label deteksi pothole.

Model memakai dua roda berjari-jari 0,35 m, jarak sumbu 1,16 m, dan joint kemudi depan. Gravitasi, pitch, kontak roda dengan jalan, dan gerak vertikal dihitung oleh mesin fisika Gazebo. Plugin C++ menambahkan gaya propulsi, bantuan gaya lateral, torsi keseimbangan, dan kendali arah. Bantuan ini sengaja membuat sepeda mudah dikendalikan pengunjung, termasuk saat berhenti. Batas kecepatan maju mendekati 25 km/jam; demonstrasi paling jelas sekitar 10–12 km/jam.

## IMU dan GPS

IMU Gazebo menghasilkan percepatan dan orientasi pada 100 Hz dengan noise akselerometer 0,045 $m/s^2$. Percepatan diputar ke kerangka dunia menggunakan quaternion orientasi, lalu gravitasi 9,81 $m/s^2$ dikurangkan. Bias statis diperkirakan saat awal kalibrasi. Filter low-pass meredam noise sebelum pengukuran amplitudo, Z-DIFF, dan STDEV pada jendela 20 sampel.

Kandidat dimulai ketika percepatan tersaring melewati 1,2 $m/s^2$. Arah awal dikonfirmasi oleh perubahan pitch dari orientasi IMU: bagian depan turun >0,008 rad mendukung pothole, sedangkan bagian depan naik terlebih dahulu menolak kandidat sebagai tonjolan. Getaran paving tanpa perubahan pitch yang cukup tidak diklasifikasikan sebagai pothole. Ini mengurangi pengaruh impuls kontak yang terlewat atau teralias saat sampling.

Kandidat turun juga memerlukan simpangan perkiraan dari integral akselerasi $<-1,2$ cm, impuls negatif >0,12 m/s, benturan positif >2 $m/s^2$, serta puncak amplitudo >4,5 $m/s^2$ dalam 1,4 detik. Kandidat berakhir setelah sinyal tenang atau timeout. Integral ini hanya fitur klasifikasi jangka pendek, bukan pengukuran kedalaman lubang yang akurat. Fungsi unit-test dapat menerima akselerasi tanpa pitch, tetapi runtime Gazebo selalu menggunakan orientasi IMU.

Deteksi diizinkan setelah kalibrasi, saat kecepatan >0,7 m/s, dengan jeda antarlaporan 2,5 detik. Penyetelan ini khusus model simulator, bukan hasil validasi MPU6050 nyata. Detektor tidak membaca koordinat atau jenis objek dunia untuk memutuskan pothole. Perubahan kecepatan, geometri, model ban, atau noise dapat memerlukan penyetelan dan pengujian ulang; deteksi sempurna pada semua kondisi tidak dijanjikan.

NavSat Gazebo berjalan 5 Hz. Noise horizontal sekitar 0,35 m disetel sebagai 0,00000315 derajat karena implementasi NavSat Gazebo sensors8 menggunakan derajat untuk noise horizontal. Saat pola IMU dikonfirmasi, koordinat diinterpolasi dari riwayat GPS pada awal kandidat agar keterlambatan klasifikasi tidak memindahkan titik terlalu jauh. Celah GPS di atas 0,6 detik tidak diinterpolasi. Jika sampel tidak tersedia, sistem tidak mengarang koordinat.

## Vision

Kamera depan berada sekitar 1,08 m dari permukaan jalan, menunduk 0,30 radian, dengan horizontal field of view 1,20 radian. Resolusi 640 × 360 dan target 15 FPS. Baseline OpenCV memilih kontur gelap pada area jalan, menolak bentuk yang terlalu kecil atau tidak sesuai, lalu memperkirakan jarak dengan perpotongan sinar kamera terhadap bidang jalan. Kandidat pada rentang sekitar 0,6–3 m dapat memicu buzzer.

Ketebalan bagian dalam kontur diperiksa menggunakan distance transform dengan padding nol di batas citra. Kandidat perlu tinggi kotak minimal 12 piksel, radius bagian dalam minimal 4 piksel, dan fill factor minimal 0,45. Retakan tipis, termasuk potongan retakan di tepi bawah kamera, ditolak. Batas ini juga dapat mengurangi kemampuan mendeteksi pothole yang sangat kecil atau jauh; rentang demo tetap perlu diuji.

Detektor tidak mengakses daftar lubang, nama objek Gazebo, segmentation label, atau koordinat dari dunia. Karena memakai kontras, bayangan, tambalan aspal, atau pencahayaan lain dapat menyebabkan salah deteksi. Jarak visual merupakan perkiraan bidang datar, terutama kurang tepat ketika sepeda sedang menunduk atau memantul. Skor internal berasal dari bentuk kontur dan bukan probabilitas terkalibrasi.

## Peringatan berdasarkan database

Perangkat mengubah GPS ke koordinat lokal. Titik database harus berada di depan arah gerak dan dekat lintasan yang diperkirakan dari sudut kemudi serta jarak sumbu roda. Koridor jalur 1,6 m menoleransi galat posisi dan lebar lubang. Jarak lihat ke depan dihitung dari empat detik perjalanan, minimal 6 m dan maksimal 15 m. Sepeda harus bergerak maju setidaknya 0,35 m/s. Dengan demikian peringatan tidak hanya memakai lingkaran jarak.

Dalam prototipe ini arah dan kecepatan diperoleh dari odometri simulator. Pada perangkat fisik, bagian tersebut perlu diganti dengan estimasi heading/kecepatan dari IMU, GPS, dan/atau sensor roda. GPS saja pada kecepatan rendah tidak selalu memberikan arah yang stabil.

## Penyimpanan dan sinkronisasi

Database perangkat dan server adalah dua berkas SQLite berbeda. Setiap peristiwa mempunyai event_id unik. Perangkat menyimpan laporan sebelum mengirimnya melalui HTTP. Sinkronisasi mencoba setiap dua detik waktu nyata, dengan timeout HTTP dua detik. Laporan ditandai tersinkron hanya sesudah server memberi acknowledgment.

Server menyimpan daftar ID yang sudah diterima agar retry tidak menggandakan observasi. Laporan dengan koordinat dalam radius 2,5 m digabung sebagai satu titik; koordinat titik dihitung sebagai rata-rata observasi. Kedua lubang nyata yang sangat berdekatan dapat ikut tergabung dengan pengaturan ini. Setelah upload, perangkat mengunduh peta terbaru. Sakelar offline menghentikan percobaan sinkronisasi berikutnya; permintaan yang sudah berlangsung dapat selesai.

Reset dan siklus sinkronisasi diserialisasi untuk satu perangkat demonstrasi ini agar upload/download yang sedang berjalan tidak mengembalikan data terhapus. Server menolak sumber laporan vision; semua laporan sensor baru memakai source=imu. Marker inisialisasi persisten mencegah seed B muncul lagi setelah reset. Ini bukan protokol reset terdistribusi untuk banyak perangkat fisik sekaligus.

# 8. Berkas proyek dan parameter

| Berkas / folder | Fungsi |
|---|---|
| start.sh dan run.py | Launcher, pengawasan proses, penghentian proses milik sesi |
| simulator/BicycleSystem.cc | Plugin gaya gerak, keseimbangan, kemudi, dan odometri |
| scripts/generate_world.py | Pembuatan SDF, geometri jalan, sepeda, tekstur dan lingkungan |
| config/demo.json | Parameter IMU, vision, GPS acuan, peringatan, dan lokasi lubang dunia |
| pothole/core.py | Deteksi IMU, interpolasi GPS, dan logika peringatan arah/jalur |
| pothole/track.py | Geometri oval untuk peta dan pengikut lintasan, bukan detektor |
| pothole/vision.py | Detektor citra yang dapat diganti dengan model FOMO |
| pothole/runtime.py | Penghubung sensor Gazebo dengan logika perangkat |
| pothole/storage.py dan cloud.py | Database persisten, deduplikasi, server HTTP |
| web/ | Tampilan dashboard, keyboard, Gamepad API, dan suara buzzer |
| data/[sesi]/ | device.sqlite3, server.sqlite3, events.jsonl, dan logs/ |
| generated/ dan build/ | Aset dan plugin hasil pembuatan otomatis |

Setelah mengubah geometri atau parameter sensor, hentikan lalu jalankan kembali launcher. Aset dibuat ulang agar path lokal tetap benar setelah folder dipindahkan. Jangan mengedit aset generated/ sebagai sumber utama; ubah generator atau konfigurasi.

Endpoint lokal perangkat: GET /api/state, GET /snapshot/camera.jpg, GET /snapshot/chase.jpg, GET /api/export.csv, POST /api/control, POST /api/options, POST /api/reset, dan POST /api/database/reset. Reset database memerlukan JSON confirm="RESET POTHOLE DATABASE"; gunakan tombol dashboard untuk operasi normal. Endpoint server: GET /api/potholes, POST /api/reports, dan POST /api/database/reset. Server demo hanya bind 127.0.0.1; autentikasi, TLS, hak akses, dan deployment publik belum menjadi bagian aplikasi ini.

# 9. Memasang pada komputer lain

Gunakan Ubuntu yang didukung Gazebo Harmonic. Lingkungan pengerjaan adalah Ubuntu 22.04, Gazebo 8.15, dan Python 3.10. Ikuti dokumentasi resmi Harmonic untuk memasang paket Gazebo beserta repository-nya. Dependensi tambahan:

```
sudo apt install build-essential cmake pkg-config \
  python3-numpy python3-opencv python3-pil \
  python3-reportlab python3-pytest
sudo apt install libgz-sim8-dev libgz-plugin2-dev \
  python3-gz-transport13 python3-gz-msgs10
```

Salin folder proyek termasuk kode, konfigurasi, dan dokumentasi. Launcher membuat ulang build/aset lokal. Gunakan Python sistem melalui start.sh; aktivasi ROS atau virtualenv tidak dibutuhkan. GPU dengan dukungan OpenGL/EGL disarankan. Kebutuhan CPU/GPU untuk mencapai realtime harus diperiksa pada laptop pagelaran, karena tingkat bayangan dan dua kamera menambah biaya rendering.

# 10. Pemecahan masalah

| Gejala | Pemeriksaan dan tindakan |
|---|---|
| Port sedang digunakan | Hentikan launcher lama dengan Ctrl+C atau gunakan pasangan port lain. Jangan menjalankan dua demo pada port sama. |
| Dashboard hidup, gambar belum muncul | Periksa logs/gazebo.log dan indikator CAMERA. Tunggu pemuatan awal; coba --headless jika layar tidak tersedia. |
| EGL/OpenGL gagal | Periksa driver GPU. Coba mode biasa tanpa --headless pada desktop. Rendering software dapat berjalan lebih lambat. |
| Sepeda tidak bergerak | Tunggu kalibrasi, klik area gambar/judul, pastikan jendela fokus, dan periksa sensor GAZEBO/IMU. |
| Tombol W mengubah menu | Fokus masih pada select/tombol. Klik gambar lalu coba lagi. |
| Buzzer tidak terdengar | Klik Aktifkan suara, periksa volume/mute browser dan output speaker. LED dan label sumber tetap memperlihatkan status. |
| Stik belum terdeteksi | Hubungkan di OS, tekan tombol setelah membuka halaman localhost, lalu periksa indikator. Coba kabel USB dan browser lain. |
| Antrean tidak habis | Aktifkan Jaringan online; lihat error sinkronisasi dan logs/server.log. Periksa port server. Data lokal tetap tersimpan. |
| Titik dari demo sebelumnya muncul | Klik Reset database lalu konfirmasi. Alternatifnya, gunakan --data-dir baru. Restart saja tidak menghapus temuan. |
| Import protobuf gagal | Jalankan melalui start.sh yang memilih Python sistem dan menghindari konflik paket pengguna. |
| Gerak atau video lambat | Tutup aplikasi berat, hindari membuka jendela Gazebo tambahan, dan gunakan GPU. Bandingkan waktu simulasi dengan waktu nyata. |

# 11. Pengujian dan batas interpretasi

Uji logika dijalankan dengan /usr/bin/python3 -s -m pytest -q. Uji arena memakai scripts/validate_oval.py terhadap sesi Gazebo terpisah; skrip tersebut menggerakkan sepeda dan sengaja mereset database uji, sehingga jangan diarahkan ke sesi data utama. Hasil terbaru dan batas pengujian tersedia pada Hasil_Pengujian_Oval.md. Pengujian versi lintasan lurus sebelumnya bukan bukti kelulusan versi oval.

Pada pengujian 22 Oktober 2026, 24 uji logika, 12 pemeriksaan integrasi, dan 8 pemeriksaan browser lulus. Sepeda menyelesaikan loop dan memberi peringatan database pada putaran berikutnya. Reset lewat tombol serta restart dengan folder data sama mempertahankan database kosong. Uji gamepad memakai input sintetis browser; stik PS fisik tetap perlu dicoba sebelum pagelaran.

Jangan menyamakan keberhasilan demo dengan pemenuhan target perangkat pada Tabel 6.2. Target sensitivitas IMU ≥80 persen, F1 vision ≥77 persen, response buzzer ≤500 ms, dan umur baterai lima jam memerlukan pengujian perangkat fisik serta dataset yang sesuai. Demo ini belum mengukur keterlambatan sampai speaker nyata, konsumsi energi, akurasi FOMO, atau kondisi cuaca nyata.

Untuk memasukkan FOMO, sediakan model terlatih beserta input shape, normalisasi, label, threshold, dan format output. Ganti Detector.detect pada pothole/vision.py untuk menghasilkan kandidat dari

piksel; pisahkan threshold dan preprocessing dari antarmuka dashboard. Validasi pada citra kamera nyata sebelum menyebut hasil sebagai performa produk akhir.

## 12. Persiapan hari pagelaran

Sehari sebelumnya, uji pada laptop, layar, kabel stik, dan speaker yang akan digunakan. Simpan satu folder data untuk latihan dan satu nama folder baru untuk demonstrasi. Buka dashboard, aktifkan suara, dan jalankan skenario A, B, serta offline D. Buka PDF panduan secara lokal agar tetap tersedia tanpa internet.

Saat pengunjung datang, mulai dari awal lintasan atau A, jelaskan fungsi masing-masing panel, lalu berikan kendali keyboard/stik. Setelah satu demonstrasi selesai, rem dan pindah skenario. Untuk mengulang penemuan pertama, klik Reset database; untuk menjelaskan crowdsourcing, pertahankan database dan lewati titik yang telah tersimpan.

## Referensi


- B-08_Naskah Proposal C251 Final.pdf, Dokumen C-251 revisi 01, 14 Agustus 2026, terutama Tabel 6.2 serta Bab 7.

- Gazebo Harmonic, Sensors: https://gazebosim.org/docs/harmonic/sensors/

- Gazebo Transport 13, Python support: https://gazebosim.org/api/transport/13/python.html

- Gazebo Sim 8, Link API: https://gazebosim.org/api/sim/8/classgz_1_1sim_1_1Link.html

- Gazebo sensors8, NavSatSensor.cc (noise horizontal dalam derajat): https://github.com/gazebosim/gz-sensors/blob/gz-sensors8/src/NavSatSensor.cc

- Pemasangan Harmonic: https://gazebosim.org/docs/harmonic/install_ubuntu/